

Logistic Regression

Github	https://github.com/jacksonjgee
Linkedin	https://www.linkedin.com/in/jacksonjgee/

Table Of Contents

[Table Of Contents](#)

[Introduction](#)

[How this works \(Mathematical Theory\)](#)

[Notation Summary](#)

[Idea of Logistic Regression](#)

[Likelihood Function](#)

[3 Main Changes](#)

[Deriving the Cost Function](#)

[Take the natural logarithm](#)

[Divide by \$m\$](#)

[Negate it](#)

[Gradient Descent](#)

[Python Implementation](#)

[Testing Our Logistic Regression Model](#)

Introduction

Logistic regression is a machine learning algorithm used for classification. Although it has the word "regression" in its name, it is mainly used to predict categories, especially in binary classification problems.

The reason it is called regression is because the model does not directly output a class label. Instead, it first outputs a probability between 0 and 1. This probability represents how likely something is to belong to a particular class.

For example, a logistic regression model could be used to classify whether an image is a cat or not a cat. If the model gives an image a probability of 0.95, this means the model is very confident that the image belongs to the "cat" class. If the model gives another image a probability of 0.05, this means the model thinks it is very unlikely to be a cat.

The model does not automatically say "Cat" or "Not Cat". To turn the probability into a final classification, we use a threshold. A common threshold is 0.5. If the predicted probability is greater than or equal to 0.5, we classify the result as 1, meaning "cat". If the predicted probability is less than 0.5, we classify the result as 0, meaning "not cat".

This makes logistic regression useful for binary classification, where there are only two possible outcomes.

How this works (Mathematical Theory)

Notation Summary

Use this notation summary as a reference while reading. If you forget what a symbol means, you can come back here at any time. Please note that this document uses standard ML notation.

- m is the number of training examples/instances.
- n is the number of features.
- X is the whole dataset of instances.
- x^i is the i -th training instance.
- x_j^i is a feature j of instance i .
- θ is the parameter vector.
- θ_j is the j -th parameter (coefficient).
- θ^T is the transposed version of the vector of parameters.
- $z^{(i)}$ is the raw score for training instance i , given by:
$$z^{(i)} = \theta^T x^{(i)}$$
- Z is the raw score for all the training instances in the dataset, given by:
$$Z = X\theta$$
- x_0 is the bias/intercept term.
- $\sigma(z^{(i)})$ is the predicted probability after the raw score $z^{(i)}$ is passed through the sigmoid function, given by:

$$\sigma(z^{(i)}) = \frac{1}{1+e^{-z^{(i)}}}$$

- $h_{\theta}(x^{(i)})$ is the hypothesis function. It gives the predicted probability that training example i belongs to the positive class, class 1, given by:

$$h_{\theta}(x^{(i)}) = \sigma(z^{(i)})$$

- y for the vector of all labels/classes for the dataset.
- $y^{(i)}$ is the label/class for instance i .
- $L(\theta)$ is the likelihood function given by:

$$L(\theta) = \prod_{i=1}^m [h_{\theta}(x^{(i)})]^{y^{(i)}} [1 - h_{\theta}(x^{(i)})]^{1-y^{(i)}}$$

- $\ell(\theta)$ is the log-likelihood function, given by:

$$\ell(\theta) = \ln(L(\theta))$$

- $J(\theta)$ is the cost function we are trying to minimise, given by:

$$J(\theta) = -\frac{\ell(\theta)}{m} = -\frac{\sum_{i=1}^m y^{(i)} \cdot \ln(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \cdot \ln(1 - h_{\theta}(x^{(i)}))}{m}$$

Idea of Logistic Regression

From our dataset, we have an input $x^{(i)}$, which is the i -th instance in the dataset, and is a vector that contains n features. $x_j^{(i)}$ can represent an individual feature j of instance $x^{(i)}$. In a dataset there would be m instances.

For one training example, we multiply the feature vector $x^{(i)}$ by the parameter vector θ .

$$z^{(i)} = \theta^T x^{(i)}$$

Or in simple terms:

raw score = weights $\times$ input features

or

raw score = weighted sum of input features

To get the raw score for the entire dataset we use this equation:

$$Z = X\theta$$

This represents a normal linear regression where:

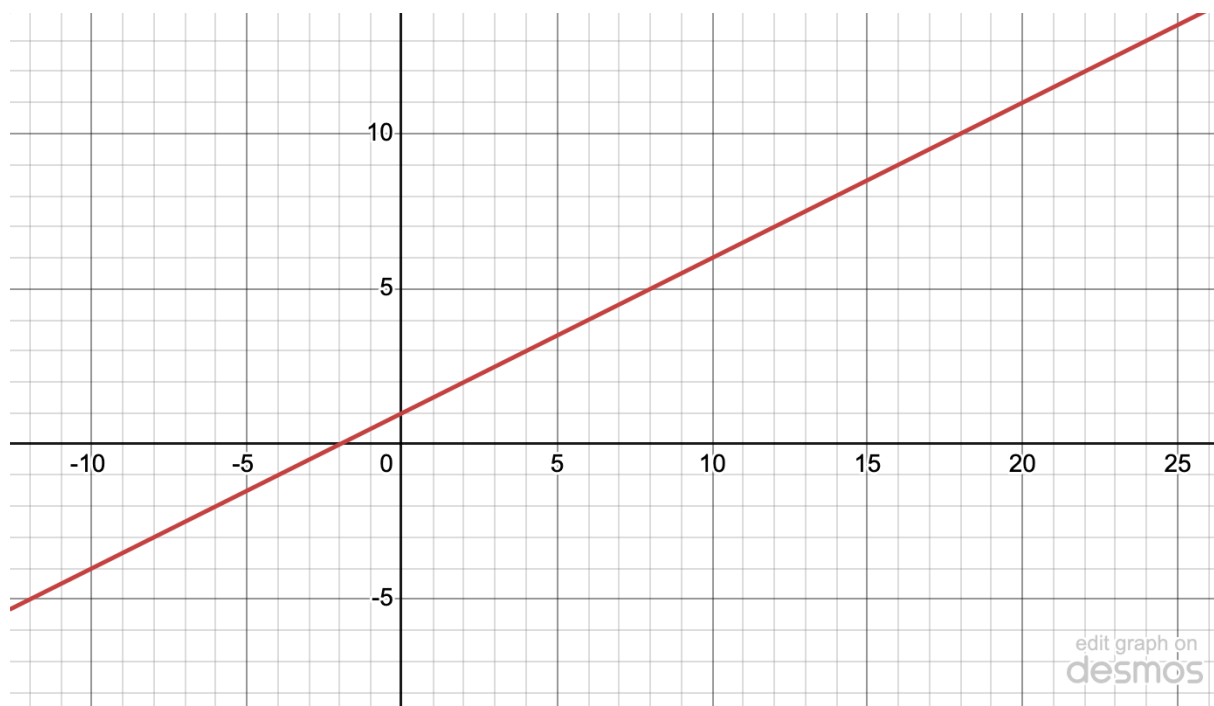
- θ is the vector of weights / parameters / coefficients.
- X is the feature matrix containing all training examples.
- Z is the vector of raw scores for all training examples.

Without the bias/intercept term, each input has n features and θ has n parameters. To include the bias/intercept, we add a constant feature $x_0 = 1$ to each input. This gives each input $n + 1$ values, and θ also has $n + 1$ parameters.

The bias/intercept controls the vertical shift of the linear function.

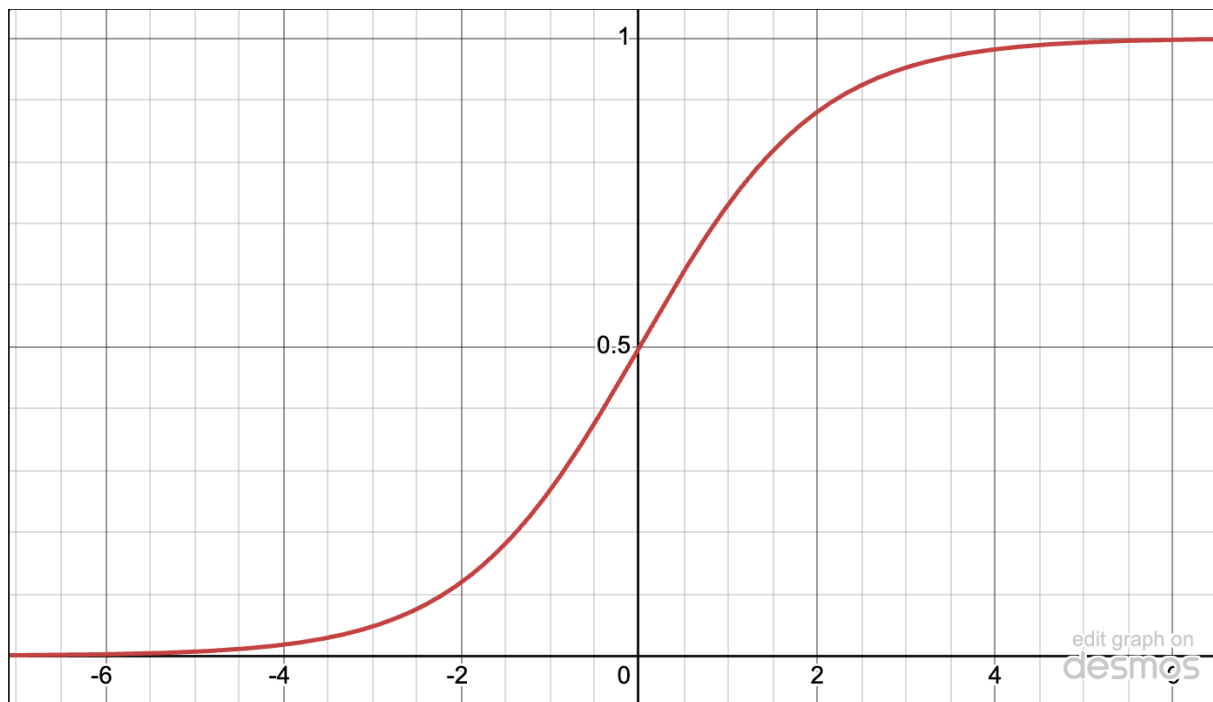
The parameters in θ are exactly what we are going to adjust, and by default they will be random values or ones - the point is that the parameters are unintelligent before training. What we are trying to do is find the right parameters in θ so that the model produces useful raw scores $z^{(i)}$. These raw scores are then passed through the sigmoid function to produce probabilities that match the true labels as closely as possible.

To do this, we first need to understand how to convert the raw score $z^{(i)}$, which can be any real number, into a value between 0 and 1 that can be interpreted as a probability. Z can look like this:



This shows that the raw score $z^{(i)}$ can increase or decrease without bound. In other words, it can approach positive infinity or negative infinity. But we need a

function that returns a value between 0 and 1. This is where the sigmoid function comes into play. Large positive raw scores are pushed close to 1, while large negative raw scores are pushed close to 0.



The Sigmoid formula we will be using is:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Where:

$$\lim_{z \rightarrow \infty} \sigma(z) = 1$$

and

$$\lim_{z \rightarrow -\infty} \sigma(z) = 0$$

This allows us to take the output $z^{(i)}$ and convert it to h , the prediction probability:

$$h_{\theta}(x^{(i)}) = \sigma(z^{(i)})$$

The prediction for instance $x^{(i)}$ is found by putting the raw model output $z^{(i)}$ through the sigmoid function. In simple terms:

$$\text{prediction probability} = \text{sigmoid}(\text{raw score})$$

The function $h_{\theta}(x^{(i)})$ gives the estimated probability that an instance belongs to the positive class, or class 1.

However, this probability alone is not enough for training the model. We cannot simply optimise $h_{\theta}(x^{(i)})$ directly, because optimising it would push every prediction closer to 1, even though not every instance belongs to the positive class. If the true answer is 1, we want the model to predict a high probability. But if the true answer is 0, we want the model to predict a low probability.

Therefore, we need a cost function that measures how wrong the model's predictions are. This gives us something to optimise, so the model can adjust its parameters and improve over time.

Likelihood Function

We can derive a cost function by thinking about it in terms of likelihood. The likelihood function measures how well a particular set of parameters θ explains the training data.

$$L(\theta) = \prod_{i=1}^m \left[h_{\theta}(x^{(i)}) \right]^{y^{(i)}} \left[1 - h_{\theta}(x^{(i)}) \right]^{1-y^{(i)}}$$

This function says: for every training instance (remember m is the number of instances or more specifically instances used for training), calculate how likely the model's prediction was to match the correct answer, then multiply all of those probabilities together.

The symbol $\prod$ means product, which is similar to how $\sum$ means sum. Instead of adding values together, we multiply them together.

The clever part of this function is that it handles both possible labels, $y = 1$ and $y = 0$, in one formula. $y = 1$ being the positive instance case, and $y = 0$ being the negative instance case. In other words, y^i is the actual correct answer for that training instance i .

If the true label is 1 for instance i , then:

$$y^i = 1$$

so the right part of the function after $\prod$ becomes:

$$h_{\theta}(x^{(i)})$$

because:

$$\left[1 - h_{\theta}(x^{(i)})\right]^{1-1} = 1$$

This means we want the model to give a high probability for class 1, since $h_{\theta}(x^{(i)})$ gives the estimated probability that an instance belongs to the positive class, or class 1.

If the true label is 0 for instance i , then:

$$y^{(i)} = 0$$

so the function becomes:

$$1 - h_{\theta}(x^{(i)})$$

because:

$$h_{\theta}(x^{(i)})^0 = 1$$

This means we want the model to give a low probability for the positive class, or class 1.

Each training instance gives us one probability showing how likely the model was to produce the correct label for that instance. By multiplying these probabilities together, we get one overall likelihood score for the whole dataset.

Therefore, the likelihood function rewards the model when it gives high probabilities to the correct labels. The better the parameters θ , the higher the likelihood will be.

SUMMARY:

The likelihood function tells us how likely the correct training labels are, given the model's current parameters. The more mistakes the model makes the smaller the value the likelihood function will produce. The goal is to find parameters that make the observed data as likely as possible.

BUT...

The likelihood function works in principle, but before we can use it as a proper cost function for training, we usually make three important changes.

3 Main Changes

1. Take the natural logarithm

The likelihood function multiplies many probabilities together. Since probabilities are between 0 and 1, multiplying many of them can produce extremely small numbers, which can become difficult for a computer to work with.

To fix this, we take the natural logarithm of the likelihood.

This also makes the maths much easier, because logarithms turn products into sums:

$$\ln(a \times b) = \ln(a) + \ln(b)$$

So instead of working with a product of many probabilities, which can become an extremely small number, we work with a sum, which is easier to calculate and optimise.

2. Divide by m

After taking the logarithm, the product becomes a sum over all training examples. However, if we have more training examples, the total cost will naturally become larger simply because there are more examples being added together.

To make the cost easier to compare across different dataset sizes, we divide by m , where m is the number of training instances.

This gives us the average cost per training example, rather than the total cost across the whole dataset.

3. Negate it

The likelihood function measures how well the model explains the training data. A higher likelihood means the model is doing better.

However, gradient descent is designed to minimise a function, not maximise it. So instead of trying to maximise the likelihood, we multiply it by -1 and turn it into something we can minimise.

This changes the idea from:

make the model as likely as possible

to:

make the model's error as small as possible

This is how we move from a likelihood function toward a cost function. If this does not fully make sense yet, do not worry. The main idea is that these three changes turn the likelihood function into a cost function that is easier to optimise. Once we apply the changes and see the final formula, the reason for each step will become clearer.

Deriving the Cost Function

We are trying to move from likelihood function to the cost function by applying those previously mentioned 3 changes.

$$L(\theta) \rightarrow J(\theta)$$

Where $J(\theta)$ is the cost function. Reminder of the likelihood function for convenience:

$$L(\theta) = \prod_{i=1}^m \left[h_{\theta}(x^{(i)}) \right]^{y^{(i)}} \left[1 - h_{\theta}(x^{(i)}) \right]^{1-y^{(i)}}$$

Take the natural logarithm

First lets take the natural logarithm of the likelihood function:

$$\ell(\theta) = \ln(L(\theta))$$

here we have to know that some rules when it comes to using logarithms:

$$\ln(a \times b) = \ln(a) + \ln(b)$$

and

$$\ln(a^b) = b \ln(a)$$

Therefore, applying these rules, taking the natural logarithm of likelihood function, we find that $\ell(\theta)$ is equal to:

$$\ell(\theta) = \sum_{i=1}^m y^{(i)} \cdot \ln(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \cdot \ln(1 - h_{\theta}(x^{(i)}))$$

This gives us the log-likelihood function, which is easier to work with because it avoids multiplying many small probabilities together, producing values extremely close to zero. Remember that we use $\sum$ because products turn to sums in logarithms.

Divide by m

To make this function scale invariant, which is a technical way of saying that the cost should be comparable no matter how many training examples we have, we divide by m , the number of training instances.

$$\frac{\ell(\theta)}{m} = \frac{\sum_{i=1}^m y^{(i)} \cdot \ln(h_{\theta}(X^{(i)})) + (1 - y^{(i)}) \cdot \ln(1 - h_{\theta}(X^{(i)}))}{m}$$

This is the average cost across all training instances. Instead of measuring the total error over the whole dataset, we are measuring the average error per example. This makes the cost easier to compare, even if the dataset size changes.

Negate it

Remember, we negate the average log-likelihood because gradient descent minimises functions. The original average log-likelihood is something we would want to maximise, so multiplying it by -1 turns it into a cost function that we can minimise.

$$J(\theta) = -\frac{\ell(\theta)}{m}$$

The full equation is:

$$J(\theta) = -\frac{\sum_{i=1}^m y^{(i)} \cdot \ln(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \cdot \ln(1 - h_{\theta}(x^{(i)}))}{m}$$

This is the final cost function for logistic regression. It measures the average error of the model's predictions across all training examples. A lower cost means the model's predicted probabilities are closer to the correct labels.

FUN FACT!

$$J(\theta) = \text{cross-entropy loss function}$$

We just calculated the cross-entropy loss function specifically used for logistic regression. This is a loss function that is often used to train neural networks. Cross-entropy is the negative average log-likelihood. It measures how well the predicted probabilities match the actual labels, giving a higher cost when the model is confidently wrong and a lower cost when the model is confidently correct.

This is the loss function that we want to minimise to perform gradient descent, finding the best parameters. Now how do we do that?

Gradient Descent

We have parameters θ that need to be adjusted in order to minimise the cost function $J(\theta)$. For each parameter θ_j , we want to know how changing that parameter affects the cost. Gradient descent uses this information to update each parameter in the direction that reduces $J(\theta)$.

We use calculus or more specifically partial derivatives to do this. A partial derivative measures how changing one variable affects a multi-variable function, while keeping the other variables constant.

$$\frac{\partial J}{\partial \theta_j}$$

This is the partial derivative notation ∂ . The above partial derivative asks: How does the function J change with respect to θ_j ? Or in better terms: how is the cross-entropy loss affected when we change this parameter θ_j ? Remember, we have n features, meaning that we have n parameters before including the bias term. θ_j is an individual parameter or the j -th parameter. This is just an example, and we will not be working with θ_j directly.

To understand how changing θ_j affects the cost, we use the chain rule. The cost function $J(\theta)$ does not depend on θ_j in one direct step. Instead, changing θ_j affects the raw score Z , the raw score affects the predicted probability $h_\theta(x^{(i)})$, the predicted probability, together with the true label y^i , affects the log-likelihood $\ell(\theta)$, and the log-likelihood affects the final cost $J(\theta)$. Use the following flow as a guide:

$$\theta_j \rightarrow z^i \rightarrow h_\theta(x^{(i)}) \rightarrow \ell(\theta) \rightarrow J(\theta)$$

So we are trying to solve:

$$\frac{\partial J}{\partial \theta_j} = \frac{\partial z^{(i)}}{\partial \theta_j} \cdot \frac{\partial h_\theta(x^{(i)})}{\partial z^{(i)}} \cdot \frac{\partial \ell}{\partial h_\theta(x^{(i)})} \cdot \frac{\partial J}{\partial \ell}$$

Start off the chain by looking at how changing a parameter θ_j affects the raw score $z^{(i)}$. In other words, we want to understand:

$$\frac{\partial z^{(i)}}{\partial \theta_j}$$

This tells us how much the raw score $z^{(i)}$ changes when we change one specific parameter θ_j , while keeping the other parameters fixed. Since the actual raw score equation is:

$$z^{(i)} = \theta^T x^{(i)}$$

Therefore the derivative of $z^{(i)}$ with respect to θ_j is:

$$\boxed{\frac{\partial z^{(i)}}{\partial \theta_j} = x_j^{(i)}}$$

Next we can look at how the predicted values $h_\theta(x^{(i)})$ changes when the raw score $z^{(i)}$ changes. Since the prediction function is the sigmoid function:

$$h_\theta(x^{(i)}) = \sigma(z^{(i)})$$

and the sigmoid function is:

$$\sigma(z^{(i)}) = \frac{1}{1 + e^{-z^{(i)}}}$$

We want to find:

$$\frac{\partial h_\theta(x^{(i)})}{\partial z^{(i)}}$$

This tells us how much the predicted probability changes when the raw score $z^{(i)}$ changes. You can derive this manually, however the derivative to the sigmoid function is a well-known derivative which we can use as follows:

$$\boxed{\frac{\partial h_{\theta}(x^{(i)})}{\partial z^{(i)}} = h_{\theta}(x^{(i)})(1 - h_{\theta}(x^{(i)}))}$$

Next we can look at how the log-likelihood $\ell(\theta)$ changes when the predicted probability $h_{\theta}(x^{(i)})$. Since the log-likelihood equation is:

$$\ell(\theta) = \sum_{i=1}^m y^{(i)} \cdot \ln(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \cdot \ln(1 - h_{\theta}(x^{(i)}))$$

Now deriving $\ell(\theta)$ with respect to $h_{\theta}(x^{(i)})$ is a bit complicated and out of the scope of these notes, however you are welcome to derive it by-hand on your own or by using a computational engine. The derivative of $\ell(\theta)$ with respect to $h_{\theta}(x^{(i)})$ is:

$$\boxed{\frac{\partial \ell(\theta)}{\partial h_{\theta}(x^{(i)})} = \frac{y^{(i)}}{h_{\theta}(x^{(i)})} - \frac{1 - y^{(i)}}{1 - h_{\theta}(x^{(i)})}}$$

Finally, we can look at how the cross-entropy loss is affected when the log-likelihood changes. Since the cross entropy loss function is:

$$J(\theta) = -\frac{1}{m} \cdot \ell(\theta)$$

The partial derivative of the cross-entropy loss function with respect to the log-likelihood is:

$$\boxed{\frac{\partial J}{\partial \ell} = -\frac{1}{m}}$$

Now that we have found how each part of the chain, we can combine them using the chain rule. We multiply these individual rates of change together to find how the cost changes with respect to one parameter θ_j .

$$\frac{\partial J}{\partial \theta_j} = \frac{\partial z^{(i)}}{\partial \theta_j} \cdot \frac{\partial h_{\theta}(x^{(i)})}{\partial z^{(i)}} \cdot \frac{\partial \ell}{\partial h_{\theta}(x^{(i)})} \cdot \frac{\partial J}{\partial \ell}$$

Which is:

$$\frac{\partial J}{\partial \theta_j} = x_j^{(i)} \cdot h_\theta(x^{(i)})(1 - h_\theta(x^{(i)})) \cdot \left(\frac{y^{(i)}}{h_\theta(x^{(i)})} - \frac{1 - y^{(i)}}{1 - h_\theta(x^{(i)})} \right) \cdot \left(-\frac{1}{m} \right)$$

Simplified is:

$$\boxed{\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right) \cdot x_j^{(i)}}$$

This means:

parameter update signal = feature value $\times$ prediction error

Where:

$$h_\theta(x^{(i)}) - y^{(i)}$$

is the model's error for that training instance. For the full dataset, we sum this over all training examples:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m \left(h_\theta(x^{(i)}) - y^{(i)} \right) \cdot x_j^{(i)}$$

Which is the final derivative, but we are more interested in using the gradient notation of this equation:

$$\nabla_\theta J(\theta) = \frac{1}{m} X^T (h_\theta(X) - y)$$

This is our gradient, and gives us the direction of steepest ascent for all the parameters. However, since we are doing gradient descent, we need to negate it and move in the opposite direction of ascent. So for every parameter (the vector of the parameters) we do this:

$$\theta := \theta - \alpha \cdot \nabla_\theta J(\theta)$$

Where θ is the vector of parameters, and α is the learning rate, which controls how large each update step is.

Gradient descent repeatedly calculates the gradient, then adjusts the parameters in the opposite direction of the gradient. This gradually reduces the

cost function $J(\theta)$, meaning the model's predicted probabilities become closer to the true labels.

In Layman's terms:

$$\text{new parameters} = \text{old parameters} - \text{learning rate} \times \text{gradient}$$

We repeat this process until the cost stops decreasing significantly, meaning the model has found parameters that fit the training data well.

Python Implementation

Now that we understand the theory behind logistic regression, we can start implementing it in Python. Since we have already derived the gradient, we can use it directly to update the model's parameters through gradient descent. This means we do not need to implement the likelihood function itself. Instead, we can focus on translating the key mathematical steps into code.

In your IDE of choice, make sure you have `numpy` and `scikitlearn` installed:

```
pip install numpy
pip install scikit-learn
```

Copy the above in the terminal. Then import numpy. We will import Scikit-Learn/sklearn later.

```
import numpy as np
```

Let's start by implementing the sigmoid function. Remember that the sigmoid equation is:

$$\sigma(z^{(i)}) = \frac{1}{1 + e^{-z^{(i)}}}$$

```
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

# np.exp(x) calculates e raised to the power of x
```

This function takes in a raw score, $z^{(i)}$, and converts it into a value between 0 and 1. This is what allows logistic regression to output a probability instead of an unrestricted linear value.

Next, we calculate the gradient. This tells us the direction and size of the update we should apply to our parameters. Remember that the gradient calculation function is:

$$\nabla_{\theta} J(\theta) = \frac{1}{m} X^T (h_{\theta}(X) - y)$$

```
def calculate_gradient(theta, X, y):
    m = y.size # m is the number of instances
    return (X.T @ (sigmoid(X @ theta) - y)) / m
# @ is used for matrix multiplication
```

In this function, `X @ theta` calculates the raw score for each training example. Remember the equation for calculating the raw scores:

$$\theta^T x^{(i)} = z^{(i)}$$

The sigmoid function then converts those raw scores into predicted probabilities $h_{\theta}(X)$. We subtract y from these predictions to calculate the model's prediction error, and then multiply by X^T to calculate the gradient for each parameter.

Now that we have the gradient, we need to perform gradient descent. Remember that the gradient descent adjusts our parameters using this equation:

$$\theta := \theta - \alpha \cdot \nabla_{\theta} J(\theta)$$

```
def gradient_descent(X, y, alpha = 0.1, epochs = 100, tol = 1e-7):
    X_b = np.c_[np.ones((X.shape[0], 1)), X] # Adds bias/intercept
    theta = np.zeros(X_b.shape[1]) # Parameters initialised to 0's
```



```

    for i in range(epochs):
        grad = calculate_gradient(theta, X_b, y) # Calculate the gradient
        theta -= alpha * grad # Adjust parameters

        if np.linalg.norm(grad) < tol: # tolerance break
            break

    return theta # Return the trained parameters

```

In this function, we first add a bias column to X , allowing the model to learn an intercept term. We then initialise all parameters in θ to zero. For each epoch, we calculate the gradient and update the parameters by moving in the opposite direction of the gradient. The value of `alpha` is the learning rate and controls the size of each update step.

The tolerance value, `tol`, is used as an early stopping condition. If the size of the gradient becomes insignificant, the loop stops early.

The `gradient_descent()` function returns the trained parameters, θ . These parameters can then be used to calculate predicted probabilities. The predicted probabilities will be values between 0 and 1. However, to make a final classification, we need to apply a threshold that converts each predicted probability into a class label.

```

def prediction_probability(X, theta):
    X_b = np.c_[np.ones((X.shape[0], 1)), X]
    return sigmoid(X_b @ theta)

def predict_class(X, theta, threshold = 0.5):
    class_pred = (prediction_probability(X, theta) >= threshold)
    return class_pred.astype(int)

```

The `prediction_probability()` function uses the trained parameters, `theta`, to calculate the model's predicted probabilities. We add the bias column again because the model was trained with a bias/intercept term, so the input data must have the same structure when making predictions.

The `predict_class()` function then converts these probabilities into class labels. If the predicted probability is greater than or equal to the threshold, the model predicts class 1. If it is below the threshold, the model predicts class 0. By default, we use a threshold of 0.5, meaning probabilities of 50% or higher are classified as the positive class.

Testing Our Logistic Regression Model

Now that we have implemented the main parts of logistic regression, we need to test whether our model actually works.

To do this, we will use Scikit-Learn's built-in breast cancer dataset. This is a binary classification dataset, meaning the target variable only has two possible classes. This makes it suitable for testing logistic regression.

Even though we are using Scikit-Learn to load the dataset, split the data, scale the features, and calculate accuracy, we are **not** using Scikit-Learn's logistic regression model. The actual training will still be done using the functions we implemented from scratch.

First, we import the modules we need:

```
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

Next, we load the dataset, please note you do not need to use this specific data set:

```
X, y = load_breast_cancer(return_X_y=True)
```

Here, `X` contains the input features, and `y` contains the correct class labels. We then split the data into a training set and a testing set:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

The training set is used to fit the model, while the testing set is used to check how well the model performs on unseen data. Before training, we scale the

features:

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Feature scaling is important because gradient descent works better when the input features are on a similar scale. If one feature has much larger values than another, it can dominate the gradient updates and make training slower or less stable.

We use `fit_transform()` on the training data because the scaler needs to calculate the mean and standard deviation from the training set. We then use only `transform()` on the test data so that the test set is scaled using the same values learned from the training set.

```
theta = gradient_descent(X_train_scaled, y_train)
y_pred = predict_class(X_test_scaled, theta)
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

We train the model using the scaled training data and then use the learned parameters, `theta`, to make predictions on the scaled test data. Finally, we compare the predicted labels to the true labels using accuracy.
